

ساختمان داده‌ها و الگوریتم‌ها

تمرین سوم - درخت‌ها و جست‌وجوی الگو

صادق صمدی، عرفان میرشمس
تاریخ تحویل: ۱۴۰۴/۰۸/۲۹

نمره ۱۵

۱. بهارستان

دانشگاه تهران یک سامانه جدید احراز هویت بهارستان را برای ورود دانشجویان راه‌اندازی کرده است. این سامانه فعلاً در مرحله اولیه است و هنوز تمهیدات امنیتی جدی برای آن در نظر گرفته نشده است. در حال حاضر رمز عبور کاربران این سامانه فقط از ۶ نوع کاراکتر مختلف A تا F تشکیل می‌شود.

ما از پایگاه داده سامانه، فراوانی وقوع هر یک از این کاراکترها در رمزهای عبور موجود را استخراج کرده‌ایم. هدف این است که براساس این داده‌ها، به طراحان سامانه کمک کنیم تا بتوانند رمزگذاری کارآمدتر و در نتیجه امنیت بالاتری ایجاد کنند. جدول فراوانی به صورت زیر است:

کاراکتر	A	B	C	D	E	F
فراوانی	۲۰	۱۰	۲۵	۱۵	۲۰	۱۰

۱. با استفاده از فراوانی‌های جدول بالا، درخت هافمن مناسب را بسازید. (نکته: در هر مرحله اگر بیش از دو کاراکتر وزن برابر داشتیم و ناچار به انتخاب بین چند مورد بودیم، کاراکترهایی را انتخاب کنید که ASCII کوچکتری دارند.)

۲. کد دودویی حاصل برای هر کاراکتر را استخراج کرده و طول کد هر نماد را مشخص کنید.

۳. فرض کنید در پایگاه داده فعلی در مجموع ۱۰۰۰۰ کاراکتر از این رمزهای عبور ذخیره شده است. محاسبه کنید:

(آ) اگر از کدگذاری ثابت طول (هر کاراکتر ۳ بیت) استفاده کنیم، چند بیت لازم می‌شود؟

(ب) اگر از کدهای حاصل از هافمن در بخش (۱) استفاده کنیم، چند بیت لازم خواهد بود؟

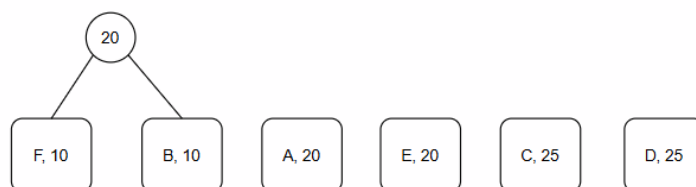
(ج) کدام روش بهینه‌تر است و چه مقدار صرفه‌جویی حاصل می‌شود؟

پاسخ:

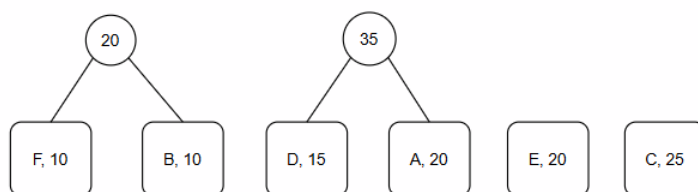
۱. وضعیت اولیه:



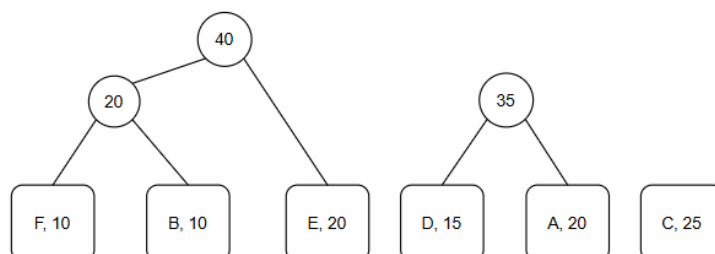
مرحله اول:



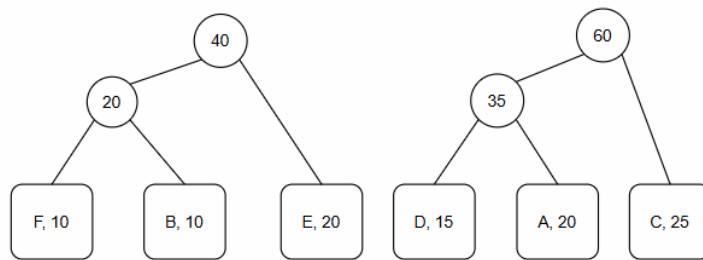
مرحله دوم:



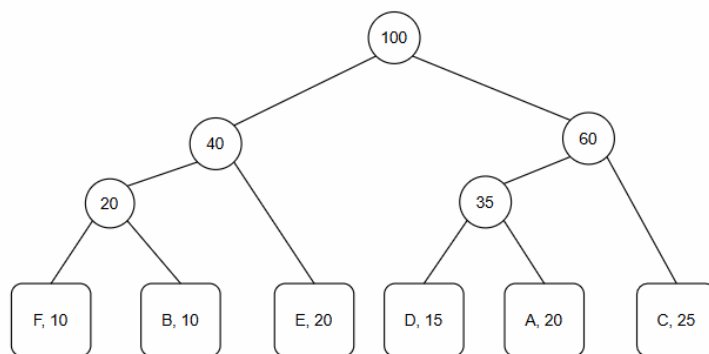
مرحله سوم:



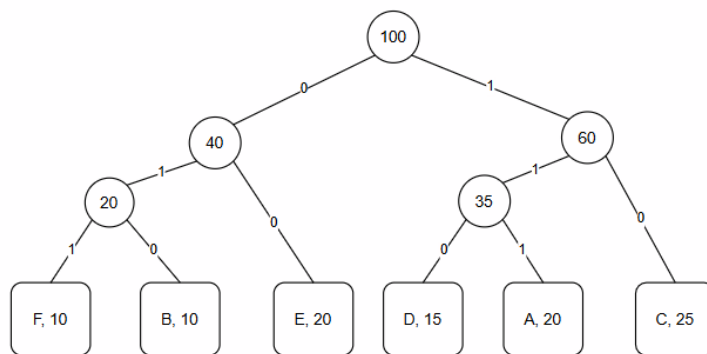
مرحله چهارم:



مرحله پنجم:



وزن دهی یال‌ها:



کدگذاری را با قراردادی ثابت انجام می‌دهیم: در هر ادغام زیردرخت اول وزن 0 و دیگری 1 می‌گیرد؛ در وزن‌های برابر، گرهی که با ASCII کوچک‌تر انتخاب شده در سمت 0 می‌نشیند.

۲. از درخت حاصل، کدها به دست می‌آیند:

نماد	کد هافمن	طول
E	۰۰	۲
B	۰۱۰	۳
F	۰۱۱	۳
C	۱۰	۲
D	۱۱۰	۳
A	۱۱۱	۳

$$|E| = |C| = 2, |A| = |B| = |D| = |F| = 3$$

۳. (آ) کد با طول ثابت (۳ بیت برای هر کاراکتر):

$$10,000 \times 3 = 30,000 \text{ بیت}$$

(ب) کد هافمن (با فراوانی‌های درصدی جدول: $\{20, 10, 25, 15, 20, 10\}$):

$$\begin{aligned} \text{بیت‌ها} &= 10,000 \times (0.20 \times 3 + 0.10 \times 3 + 0.25 \times 2 + 0.15 \times 3 + 0.20 \times 2 + 0.10 \times 3) \\ &= 10,000 \times 2.55 = 25,500 \text{ بیت} \end{aligned}$$

(ج) صرفه‌جویی:

$$30,000 - 25,500 = 4,500 \text{ بیت } (\approx 15\%).$$

۲. KMP

۱۵ نمره

صادق سر جلسه میان‌ترم درس «ساختمان داده و الگوریتم‌ها» نشسته و به سؤال مربوط به الگوریتم KMP رسیده است. اما از بدشانسی، دیشب فرصت نکرده بود این مبحث را مرور کند و الآن تقریباً کل الگوریتم از ذهنش پریده!

شما دقیقاً صندلی کناری صادق نشسته‌اید. صادق خیلی آهسته خم می‌شود و پنهانی به شما می‌گوید: «اگر این سه بخش KMP رو حل کنی، من از روی برگه‌ات می‌نویسم و یک قهوه مهمون من!»

$$P = \text{AABAAB} \quad T = \text{ABAABAABBAABAAB}$$

الف) جدول LPS مربوط به الگوی P را محاسبه کنید.

ب) الگوریتم KMP را روی رشته T اجرا کرده و تمام محل‌هایی که تطابق کامل P در T اتفاق می‌افتد را مشخص کنید.

ج) تعداد مقایسه‌های کاراکتری انجام‌شده در مرحله (ب) را بنویسید.

د) بعد از اینکه پاسخ بخش‌های بالا رو نوشتید، آیا به صادق تقلب می‌رسونید؟

پاسخ:

الف) برای $P = \text{AABAAB}$ ، جدول LPS به صورت زیر است

$$\text{LPS} = [0, 1, 0, 1, 2, 3]$$

LPS[i]	P[i]	i
0	A	0
1	A	1
0	B	2
1	A	3
2	A	4
3	B	5

ب) اجرای KMP روی T را طبق جدول زیر انجام دهید. در هر سطر وضعیت قبل از مقایسه، نتیجه مقایسه و اقدام بعدی آمده است. (نمایه‌ها صفرمینا هستند.)

$$\text{LPS}(P) = [0, 1, 0, 1, 2, 3]$$

j'	i'	اقدام	$(\neq / =)$	$P[j]$	j	$T[i]$	i	#
1	1	هر دو جلو	=	A	0	A	0	1
0	1	$j \leftarrow 0$	$\neq$	A	1	B	1	2
0	2	$i \leftarrow i+1$	$\neq$	A	0	B	1	3
1	3	هر دو جلو	=	A	0	A	2	4
2	4	هر دو جلو	=	A	1	A	3	5
3	5	هر دو جلو	=	B	2	B	4	6
4	6	هر دو جلو	=	A	3	A	5	7
5	7	هر دو جلو	=	A	4	A	6	8
3	8	Match در $i - 5 = 3$ ؛ $j \leftarrow 3$	=	B	5	B	7	9
0	8	$j \leftarrow 0$	$\neq$	A	3	B	8	10
0	9	$i \leftarrow i+1$	$\neq$	A	0	B	8	11
1	10	هر دو جلو	=	A	0	A	9	12
2	11	هر دو جلو	=	A	1	A	10	13
3	12	هر دو جلو	=	B	2	B	11	14
4	13	هر دو جلو	=	A	3	A	12	15
5	14	هر دو جلو	=	A	4	A	13	16
3	15	Match در $i - 5 = 9$ ؛ $j \leftarrow 3$	=	B	5	B	14	17

بنابراین محل‌های شروع تطابق کامل (مبنای صفر):

۹ و ۲

ج) تعداد مقایسه‌های کاراکتری انجام شده برابر است با تعداد ردیف‌های جدول بالا:

۱۷

د) خیر. کمک به تقلب خلاف اصول امانت‌داری و عدالت آموزشی است؛ بهترین کار این است که پس از آزمون، مراحل حل را با هم مرور کنیم تا یادگیری رخ بدهد.

۱۵ نمره

۳. رابین هوود

رابین هوود پس از این که از خزانه ناتینگهام کمی طلا برداشت، متوجه شد که فقط پول کافی نیست؛ دانش هم باید به فقرا برسد! او در مسیر برگشت، دفتر یادداشت یکی از ریاضی‌دان‌های دربار را پیدا می‌کند و در آن الگوریتم Rabin-Karp را می‌بیند. رابین هوود این الگوریتم را دزدیده و اسم خودش را برایش می‌گذارد: Rabin-Hood؛ تا با آن به مردم عادی یاد بدهد چگونه pattern matching را سریع انجام دهند.

الگوی مورد نظر و متن اصلی به صورت زیر هستند:

$$P = \text{ABA} \quad T = \text{ABABBABAAB}$$

فرض کنید از کدگذاری زیر برای کاراکترها و هش استفاده می‌کنیم:

$$\text{Code}(A) = ۲, \quad \text{Code}(B) = ۳, \quad b = ۳, \quad m = ۲۹$$

و تابع هش به صورت زیر تعریف شده است:

$$h(s) = \left(\sum_{i=0}^{|s|-1} \text{code}(s_i) \cdot b^{|s|-1-i} \right) \bmod m$$

الف) مقدار هش مربوط به الگوی P را محاسبه کنید.

ب) مقدار هش اولین پنجره طول ۳ در T را حساب کرده و سپس با استفاده از rolling hash پنجره‌های بعدی را به ترتیب به دست آورید.

ج) تمام محل‌هایی که تطابق کامل P در T اتفاق می‌افتد را مشخص کنید.

پاسخ:

الف) با $m = ۲۹$, $b = ۳$ و کدگذاری $\text{code}(A) = ۲$, $\text{code}(B) = ۳$ برای $P = ABA$ و $k = ۳$ داریم:

$$b^۲ \equiv ۹ \pmod{۲۹}, \quad h(P) = ۲ \cdot ۹ + ۳ \cdot ۳ + ۲ \cdot ۱ = ۱۸ + ۹ + ۲ = ۲۹ \equiv ۰ \pmod{۲۹}.$$

$$\boxed{h(P) = ۰} \text{ پس}$$

ب) برای پنجره‌های طول ۳ روی T ، هش غلتان را با رابطه زیر به روزرسانی می‌کنیم:

$$h_{i+۱} = \left((h_i - \text{code}(T[i]) \cdot b^{k-1}) \cdot b + \text{code}(T[i+k]) \right) \bmod ۱۰۱,$$

که در آن $k = ۳$ و $b^{k-1} = b^۲ = ۹$. محاسبات هش هر پنجره:

$$T = ABABBABAAB$$

h_i	پنجره طول ۳	i
۰	ABA	۰
۳	BAB	۱
۵	ABB	۲
۱	BBA	۳
۳	BAB	۴
۰	ABA	۵
۲	BAA	۶
۸	AAB	۷

ج) چون $h(P) = ۱۶$ ، تطابق‌های هش در $i = ۰$ و $i = ۵$ دیده می‌شود. با مقایسه رشته‌ای برای رفع تصادم احتمالی:

$$T[۰ : ۲] = ABA = P, \quad T[۵ : ۷] = ABA = P.$$

پس اندیس‌های شروع تطابق برابرند با $\boxed{۰}$ و $\boxed{۵}$.

۴. سلطونعلی

۱۵ نمره

علی می‌خواهد یک درخت جست‌جوی دودویی (BST) بسازد تا بتواند عملیات جست‌جو، درج و حذف را روی مجموعه‌ای از اعداد انجام دهد. استاد درس ساختمان داده به او گفته است: «اگر ترتیب درج را درست انتخاب نکنی، درخت به یک لیست پیوندی تبدیل می‌شود و همه چیز کند می‌شود!»

فرض کنید علی می‌خواهد اعداد زیر را در یک BST درج کند:

$$S = \{2, 5, 8, 12, 15, 20, 25, 30, 35\}$$

(آ) اگر علی عناصر S را به ترتیب صعودی (از کوچک به بزرگ) درج کند، درخت حاصل را رسم کنید و ارتفاع آن را محاسبه کنید.

(ب) اگر علی عناصر را به ترتیب زیر درج کند، درخت حاصل را رسم کنید و ارتفاع آن را محاسبه کنید:

$$20, 8, 30, 5, 15, 25, 35, 2, 12$$

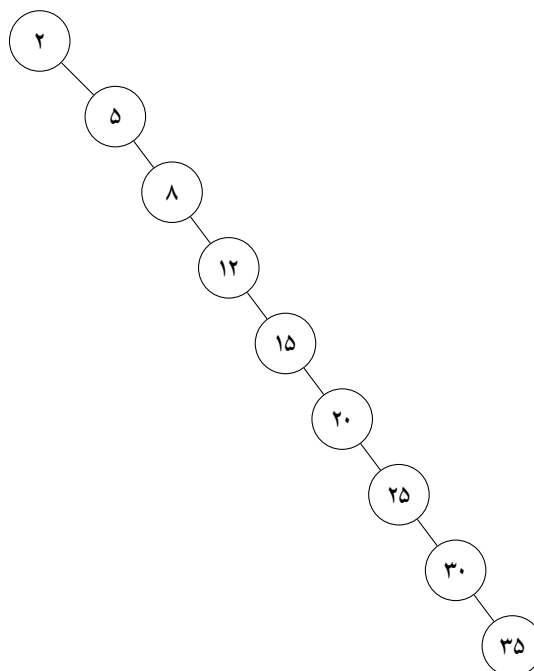
(ج) برای هر یک از درخت‌های ساخته‌شده در بخش‌های (الف) و (ب):

- تعداد مقایسه‌های لازم برای جست‌جوی عدد ۱۲ را محاسبه کنید.
- تعداد مقایسه‌های لازم برای جست‌جوی عدد ۳۵ را محاسبه کنید.
- میانگین تعداد مقایسه‌ها برای جست‌جوی یک عنصر دلخواه را تخمین بزنید.

(د) بعد از دیدن نتایج بالا، علی متوجه اهمیت ارتفاع درخت می‌شود. آیا به سخن استاد اعتقاد پیدا می‌کرد؟ چرا؟ در چه شرایطی یک BST بدترین حالت ممکن را خواهد داشت و پیچیدگی زمانی عملیات‌ها چقدر خواهد بود؟

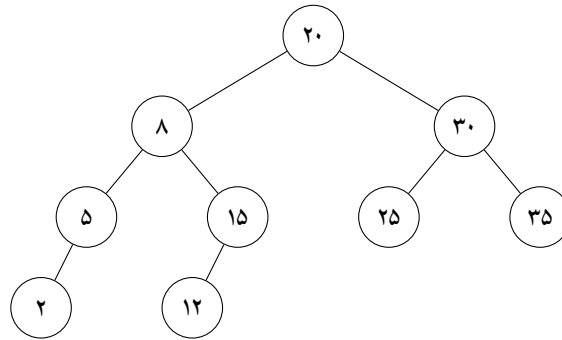
پاسخ:

(آ) درخت حاصل از درج به ترتیب صعودی یک درخت کاملاً نامتعادل (خطی) است:



ارتفاع درخت: $h = 8$ (تعداد یال‌ها از ریشه تا برگ)

ب) درخت حاصل از درج به ترتیب ۲، ۱۲، ۳۵، ۲۵، ۱۵، ۵، ۳۰، ۸، ۲۰:



ارتفاع درخت: $h = 3$

ج) مقایسه تعداد مقایسه‌ها:

درخت (الف) - نامتعادل:

- جستجوی عدد ۱۲: ۴ مقایسه (مسیر: $20 \rightarrow 8 \rightarrow 15 \rightarrow 12$)
- جستجوی عدد ۳۵: ۹ مقایسه (کل مسیر تا برگ)
- میانگین: $\frac{1+2+3+4+5+6+7+8+9}{9} = \frac{45}{9} = 5$ مقایسه

درخت (ب) - متعادل‌تر:

- جستجوی عدد ۱۲: ۴ مقایسه (مسیر: $20 \rightarrow 8 \rightarrow 15 \rightarrow 12$)
- جستجوی عدد ۳۵: ۳ مقایسه (مسیر: $20 \rightarrow 30 \rightarrow 35$)
- میانگین: تقریباً $2.78 \approx \frac{1+2+3+4+5+6+7+8}{8} = \frac{36}{8} = 4.5$ مقایسه

درخت متعادل‌تر تقریباً ۸.۱ برابر سریع‌تر عمل می‌کند!

د) بله، علی کاملاً به سخن استاد اعتقاد پیدا می‌کند. دلیل:

- درخت نامتعادل (الف) به یک لیست پیوندی تبدیل شده است
- در بدترین حالت (درج مرتب‌شده)، BST تبدیل به یک زنجیره می‌شود
- پیچیدگی زمانی عملیات‌ها در این حالت: $O(n)$
- در حالی که درخت متعادل: $O(\log n)$
- برای $n = 9$: $\log_2(9) \approx 3.17$ اما متعادل فقط ۹ مقایسه دارد اما متعادل فقط ۳.۱۷

بدترین حالت: درج داده‌های مرتب‌شده (صعودی یا نزولی) به BST، که منجر به $O(n)$ می‌شود.

۵. اسب میانه

۱۵ نمره

لطف‌علی عاشق اسب‌هاست و می‌خواهد تعدادی اسب جدید بخرد. او همیشه اسبی با قیمت میانه را انتخاب می‌کند. به این معنی که اگر k اسب در بازار موجود باشد، لطف‌علی اسبی را می‌خرد که قیمت آن در جایگاه $\lfloor \frac{k}{2} \rfloor$ قرار دارد (در صورتی که تعداد اسب‌ها زوج باشد، اسب ارزان‌تر از بین دو اسب میانی را انتخاب می‌کند).

در هر لحظه، یکی از دو عملیات زیر انجام می‌شود:

- افزودن اسب: اسبی با قیمت p به فروشگاه اضافه می‌شود.
 - خرید اسب: لطف‌علی می‌خواهد قیمت اسب میانه را بداند.
- (آ) یک ساختمان داده طراحی کنید که هر دو عملیات بالا را به‌طور کارآمد پشتیبانی کند. الگوریتم خود را توضیح دهید.
- (ب) پیچیدگی زمانی هر یک از عملیات را در ساختمان داده‌ی پیشنهادی خود تحلیل کنید.

مثال: فرض کنید عملیات‌های زیر به ترتیب انجام شوند:

- افزودن اسب با قیمت ۵
- افزودن اسب با قیمت ۲
- افزودن اسب با قیمت ۸
- خرید اسب $\rightarrow$ پاسخ: ۵ (میانه‌ی $\{2, 5, 8\}$)
- افزودن اسب با قیمت ۳
- خرید اسب $\rightarrow$ پاسخ: ۳ (میانه‌ی پایین‌تر از $\{2, 3, 5, 8\}$)

پاسخ:

(آ) ساختمان داده و الگوریتم:

از دو هرم (Heap) استفاده می‌کنیم:

- **Max Heap**: نیمه‌ی کوچک‌تر اعداد را نگه می‌دارد.
- **Min Heap**: نیمه‌ی بزرگ‌تر اعداد را نگه می‌دارد.

ویژگی اصلی:

- ریشه‌ی هرم بیشینه (بزرگترین عنصر نیمه‌ی کوچک‌تر) همیشه میانه است.
- اندازه‌ی هرم بیشینه برابر یا یکی بیشتر از اندازه‌ی هرم کمینه است.

الگوریتم افزودن اسب:

- (آ) اگر قیمت جدید از ریشه‌ی هرم بیشینه کوچک‌تر باشد، آن را به هرم بیشینه اضافه کنید، در غیر این صورت به هرم کمینه.
- (ب) اگر اختلاف اندازه‌ی دو هرم از ۱ بیشتر شد، یک عنصر را از هرم بزرگ‌تر به هرم کوچک‌تر منتقل کنید.

الگوریتم یافتن میانه:

- میانه همیشه ریشه‌ی هرم بیشینه است.

```

1 priority_queue<int> maxHeap; // smaller half (max heap)
2 priority_queue<int, vector<int>, greater<int>> minHeap;
3 // larger half
4
5 void addHorse(int price) {
6     // Add to appropriate heap
7     if (maxHeap.empty() || price <= maxHeap.top()) {
    
```

```

8         maxHeap.push(price);
9     } else {
10         minHeap.push(price);
11     }
12
13     // Balance heaps: maxHeap size = minHeap size
14     // or maxHeap size = minHeap size + 1
15     if (maxHeap.size() > minHeap.size() + 1) {
16         minHeap.push(maxHeap.top());
17         maxHeap.pop();
18     } else if (minHeap.size() > maxHeap.size()) {
19         maxHeap.push(minHeap.top());
20         minHeap.pop();
21     }
22 }
23
24 int getMedian() {
25     return maxHeap.top(); // median is always at top of max heap
26 }
    
```

ب) تحلیل پیچیدگی زمانی:

- افزودن اسب: $O(\log n)$ - افزودن به هرم و متعادل‌سازی هر دو در زمان لگاریتمی انجام می‌شوند.
 - یافتن میانه: $O(1)$ - میانه همیشه در ریشه‌ی هرم پیشینه قرار دارد.
- پیچیدگی مکانی: $O(n)$ برای نگهداری تمام قیمت‌ها در دو هرم.

۱۵ نمره

۶. پیمایش درخت

آ) پیمایش‌های Preorder و Inorder یک درخت دودویی به صورت زیر است. این درخت را رسم کرده و پیمایش Postorder آن را به دست آورید.

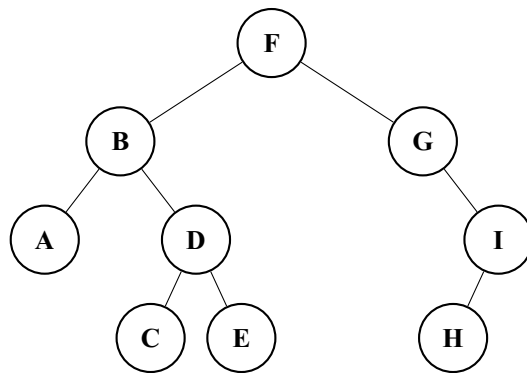
Inorder: $A, B, C, D, E, F, G, H, I$

Preorder: $F, B, A, D, C, E, G, I, H$

ب) یک الگوریتم (به صورت شبه کد) بنویسید که با دریافت دو آرایه preorder و inorder یک درخت دودویی را بازسازی کند. پیچیدگی زمانی و مکانی الگوریتم خود را تحلیل کنید.

پاسخ:

آ) از روی نمایش preorder می‌فهمیم که ریشه نود F است و از روی نمایش inorder می‌فهمیم که نودهای I و H در سمت راست و بقیه نودها در زیردرخت چپ ریشه هستند. حالاً می‌یابیم که نمایش preorder زیردرخت راست GIH و همچنین نمایش inorder آن GHI است. همچنین این نمایش‌ها برای زیردرخت چپ به ترتیب برابرند با $BADCE$ و $ABCDE$. در نتیجه ریشه زیردرخت راست G و ریشه زیردرخت چپ B است (فرزندان ریشه). این روند را برای نمایش‌های کوچک‌تری که به دست آوردیم ادامه می‌دهیم تا در نهایت به درخت زیر برسیم:



پیمایش Postorder: ابتدا زیردرخت چپ: B, A, C, E, D سپس زیردرخت راست: H, I, G و در پایان ریشه F .

$A, C, E, D, B, H, I, G, F$

ب) شبه‌کد بازسازی درخت:

```

1 def buildTree(pre, ino):
2     if not pre:
3         return None
4     # First element of preorder is root
5     rootVal = pre[0]
6     root = Node(rootVal)
7     # Find root position in inorder
8     i = ino.index(rootVal)
9     # Elements before root in inorder are left subtree
10    leftIn, rightIn = ino[:i], ino[i+1:]
11    # Split preorder accordingly
12    leftPre = pre[1:i+len(leftIn)]
13    rightPre = pre[i+len(leftIn):]
14    # Recursively build left and right subtrees
15    root.left = buildTree(leftPre, leftIn)
16    root.right = buildTree(rightPre, rightIn)
17    return root

```

تحلیل پیچیدگی:

- زمان: هر فراخوانی به ازای هر گره یک جستجوی خطی در آرایه inorder دارد: $O(n)$ ؛ و به صورت بازگشتی برای زیردرخت‌ها رجوع می‌کند. مجموع پیچیدگی $O(n^2)$ در بدترین حالت است.
- فضا: فضای بازگشتی تا عمق n در بدترین حالت (درخت زنجیری) مصرف می‌شود: $O(n)$.